Machine Learning I

Tema 2. Aprendizaje Supervisado
2.2 Redes Neuronales y Redes de Base Radial

Profesor: Raquel Martínez y Jose Tomás Palma

Curso 2024-2025

1 Teoría de la Decisión

2 Introducción a Redes Neuronales

3 Perceptrón

4 Perceptrón Multicapa

5 Redes de Funciones de Base Radial

Teoría de la decisión

La teoría de la decisión ha estado tradicionalmente asociada a los ámbitos de la economía y de la estadística y la investigación operativa. Desde que existe la Inteligencia Artificial (IA), también esta ligada con esta área. La teoría de la decisión consisten en seleccionar la mejor alternativa ante una determinada situación. Desde el punto de vista de la IA:

Tres escenarios para resolver un problema de decisión

- **Decisión analítica**. Se fundamentan en la teoría bayesiana para inferir las probabilidades a posteriori de las las clases.
- **Modelos discriminativos**. Infieren directamente las probabilidades a posteriori.
- **Funciones discriminantes**. Encuentran una función $f(\mathbf{x})$ que mapea directamente una entrada $\mathbf{x}$ a una determinada clase.

Teoría de la decisión

Decisión analítica

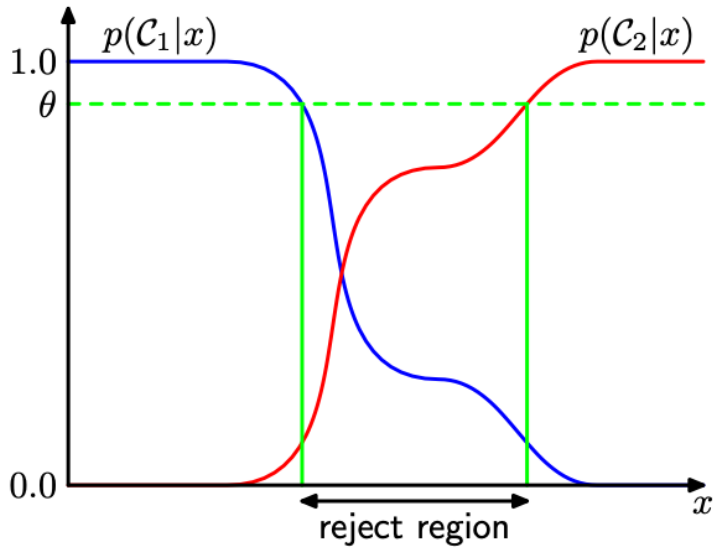
Se fundamentan en la inferencia de $p(\mathbf{x}|C_k)$ y $p(C_k)$ (o, equivalentemente $p(\mathbf{x}, C_k)$).

Procedimiento:

1. A partir de los datos, se infieren verosimilitudes y probabilidades a priori: $p(\mathbf{x}|C_k)$ y $p(C_k)$
2. Se aplica el Teorema de Bayes para calcular las probabilidades a posteriori

$$p(C_k|\mathbf{x}) = \frac{p(\mathbf{x}|C_k)p(C_k)}{p(\mathbf{x})} = \frac{p(\mathbf{x}|C_k)p(C_k)}{\sum_k p(\mathbf{x}|C_k)p(C_k)}$$

3. Se emplea la teoría de la decisión para determinar la clase de una entrada $\mathbf{x}$



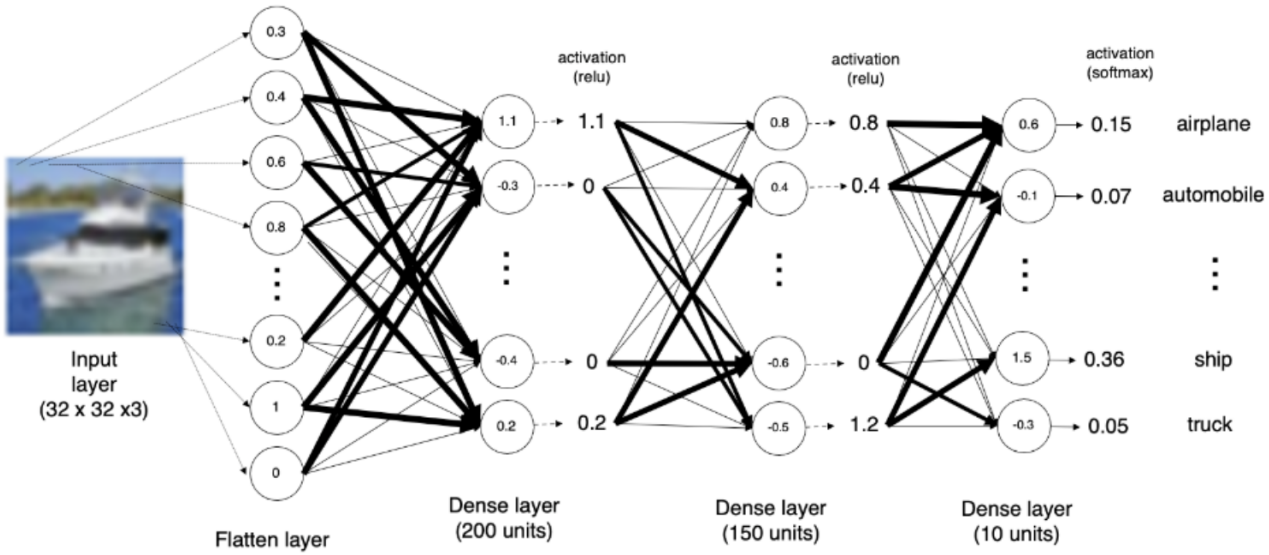
Los métodos que explícita o implícitamente modelan las distribuciones de las entradas así como las salidas, $p(\mathbf{x}, C_k)$, se llaman **modelos generativos** pues mediante *muestreo* se pueden generar datos sintéticos.

Teoría de la decisión

Modelos discriminativos

Producen directamente una estima de las probabilidades a posteriori de las clases para una entrada dada.

1. El procedimiento ahora consiste en, con los datos disponibles, diseñar y entrenar un modelo que minimice una determinada función de error entre las entradas y sus salidas deseadas.
2. Las salidas del modelos representan, mediante algún código, las probabilidades a posteriori de las clases para la entrada dada.
3. El ejemplo más claro de modelos discriminativos son las **redes neuronales progresivas** como, por ejemplo, los MLPs entrenados para minimizar la entropía cruzada con unidades de salida tipo softmax.



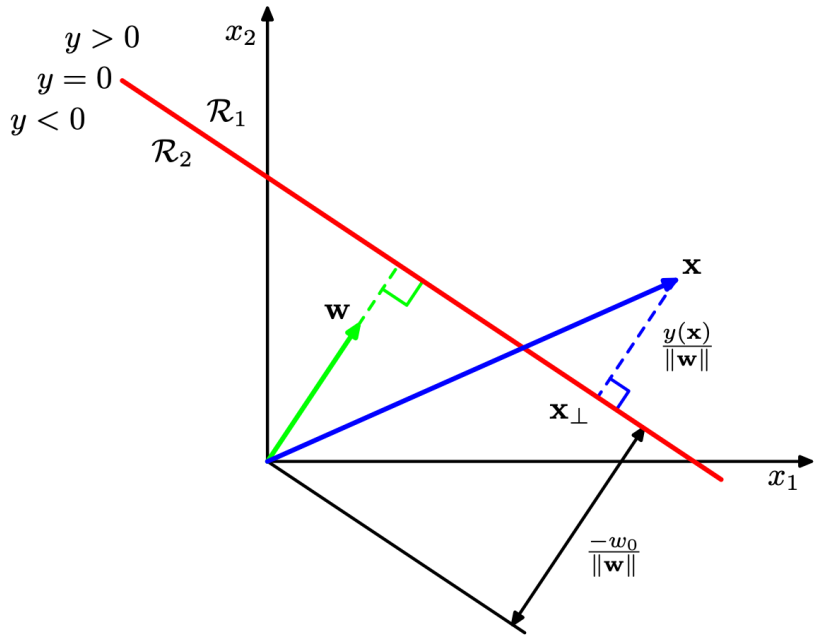
Teoría de la decisión

Funciones discriminantes

Encuentran una función $f(\mathbf{x})$, llamada *función discriminante*, que directamente mapea cada entrada $\mathbf{x}$ a la etiqueta de una clase.

- Ejemplo: problema de dos clases

$$f(\mathbf{x}) = 1, \quad \text{si} \quad y(\mathbf{x}) = \mathbf{w}^T\mathbf{x} + w_0 > 0$$
$$f(\mathbf{x}) = 0, \quad \text{si} \quad y(\mathbf{x}) = \mathbf{w}^T\mathbf{x} + w_0 < 0$$

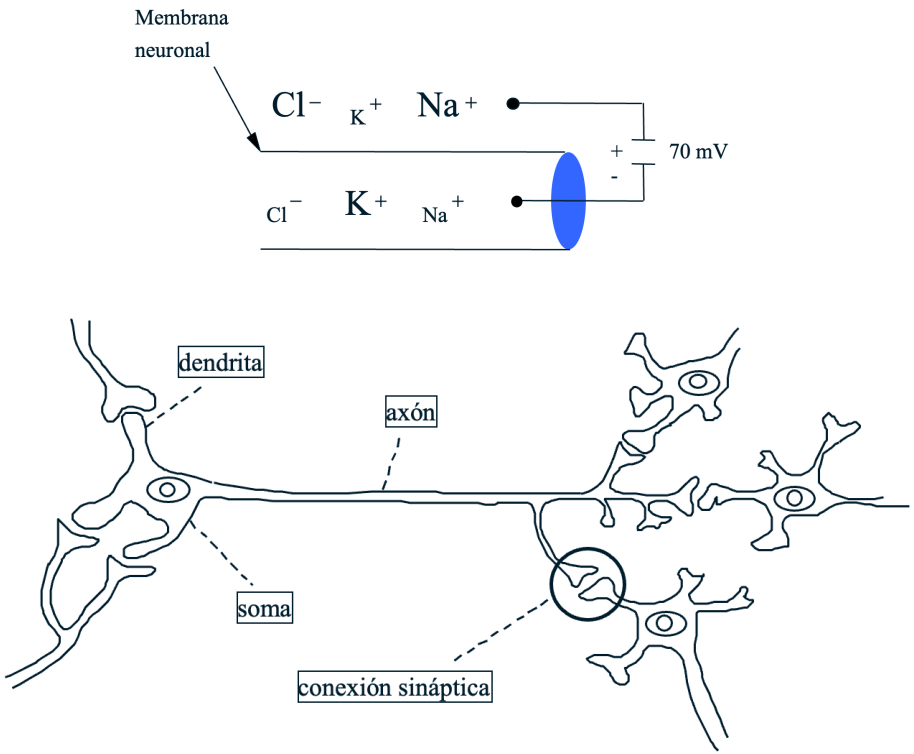


Redes Neuronales: Introducción

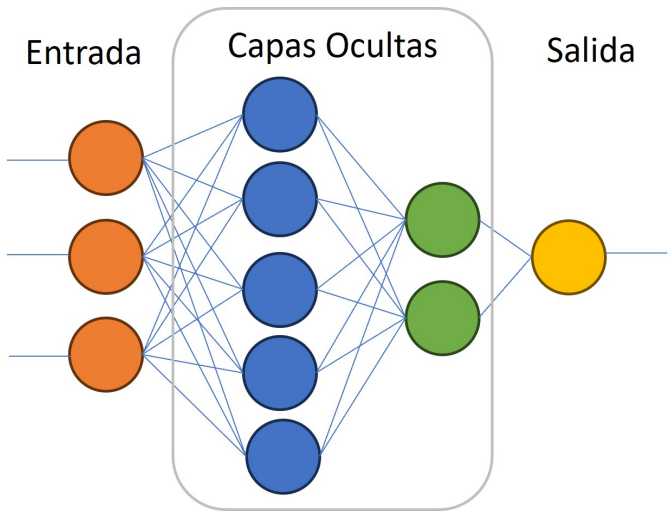
Fundamento Biológico

La redes neuronales tratan de realizar una simulación de la estructura y funcionamiento del cerebro humano para procesar información. Consisten en una serie de nodos interconectados que reciben información, la procesan y producen una salida.

Biológico



Artificial



Redes Neuronales: Introducción

Neuronas como procesadores de información sencillos. De manera simplista:

- Las dendritas constituyen el canal de entrada de la información
- El soma es el órgano de cómputo
- El axón corresponde al canal de salida, y a la vez envía información a otras neuronas.

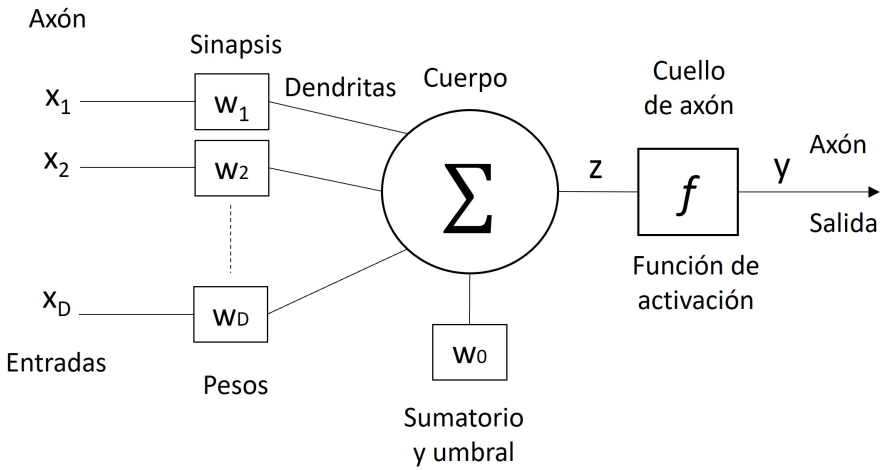
Las redes neuronales son sistemas artificiales que van a copiar la estructura de las redes neuronales biológicas para conseguir una funcionalidad similar

Redes Neuronales: Introducción

En una Red Neuronal Artificial:

- La neurona es un elemento clave.
- Las neuronas se interconectan de distintas formas dando lugar a diferentes estructuras con diferente funcionalidad.
- Cada neurona recibe un conjunto de entradas $x_1, x_2, ..., x_D$ y devuelve una única salida y.
- Existen numerosas conexiones entre distintas neuronas.
- La intensidad de la conexión viene dada por los pesos sinápticos $w_1, w_2, ..., w_D$. Por lo tanto, cada entrada de la neurona, x_i se encuentra afectada por un peso w_i
- También se suele incluir un umbral o sesgo, representado por un peso w_0 asociada a una entrada constante $x_0 = 1$.

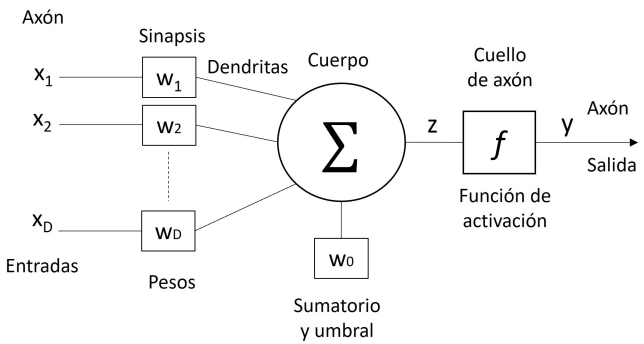
Redes Neuronales: Introducción

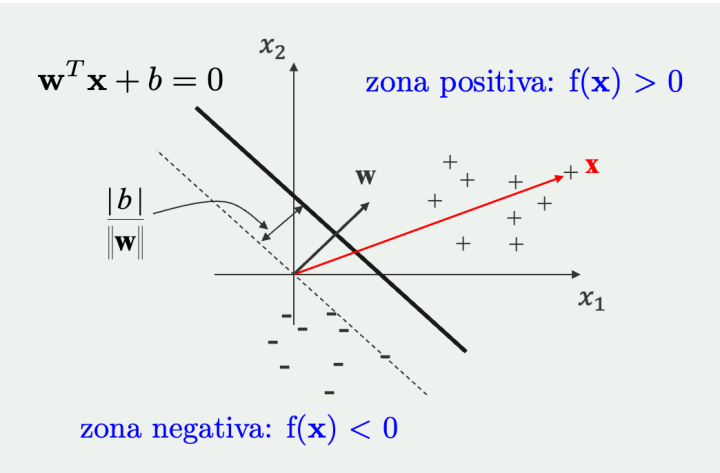


Para obtener la salida y, primero hay que calcular la activación de la neurona:

$$z = \sum_{i=1}^{D} w_i x_i + w_0 = \mathbf{w}^T \mathbf{x} + w_0$$

Redes Neuronales: Introducción





Nota: $b = w_0$

El siguiente paso consiste en aplicar a la activación la función f denominada función de activación o de transferencia:

$$y = f(z) = \sum_{i=1}^{D} w_i x_i + w_0 = g(\mathbf{w}^T\mathbf{x} + w_0)$$

En el modelo básico de McCulloch-Pitts la función de activación es la función escalón:

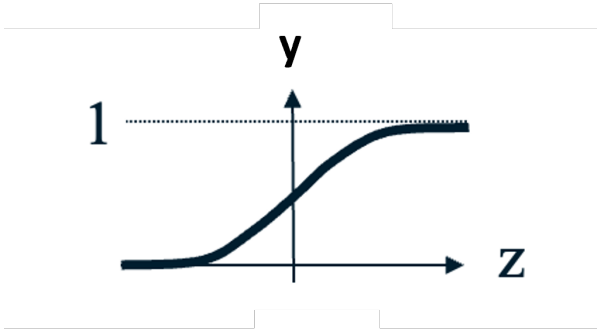
$$f(z) = \begin{cases} 1 & cuando \quad z > 0 \\ 0 & cuando \quad z < 0 \end{cases}$$

Redes Neuronales: Introducción

Algunas funciones de activación

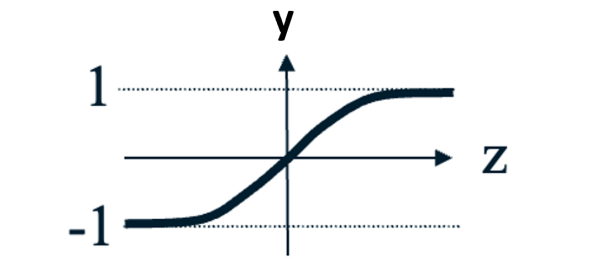
Función sigmoidea o logística:

$$y = f(z) = \frac{1}{1 + e^{-gz}}$$



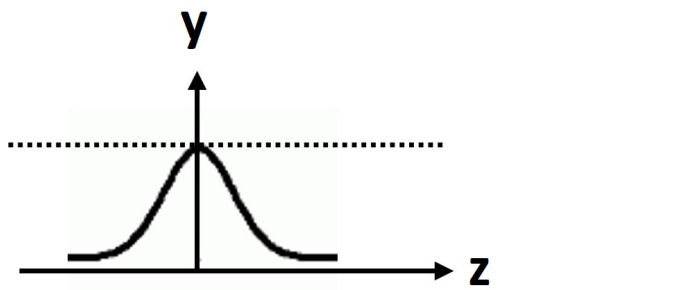
Función Tangente hiperbólica (tanh):

$$y = f(z) = \frac{1 - e^{-gz}}{1 + e^{-gz}} = th\left(\frac{gz}{2}\right)$$



Función Gausiana:

$$y = f(z) = e^{\frac{(z-\mu)^2}{2\sigma^2}}$$



g : saturación (determina la pendiente de la sigmoide en $z = 0$)
En principio, asumible por los pesos de llegada

Redes Neuronales: Introducción

Para que el modelo de McCulloch-Pitts proporcione los valores deseados es necesario someterla a un entrenamiento o aprendizaje. La regla de Hebb se basa en un hecho biológico contrastado:

- Cuando dos neuronas se activan simultáneamente su conexión se refuerza, y tiende a disminuir en caso contrario (inhibición).
- Según esto, si la entrada x_i es la salida y de otra neurona, el peso w_i se verá incrementado cuando el signo de x_i e y sea el mismo, y se decrementará cuando tengan signos contrarios:

$$w_i(k+1) = w_i(k) + \eta x_i y$$

  η es la tasa o ratio de aprendizaje y determina en qué proporción se modifican los pesos sinápticos.
- La regla de Hebb es una regla de aprendizaje no supervisada.

Perceptrón monocapa

Perceptrón Monocapa

Presentado por Rosenblatt en 1957, se considera el primer modelo de red neuronal. Está basado en el modelo de McCulloch-Pitts y la regla de Hebb. El perceptrón estaba constituido por una única neurona cuya función de activación, f, es la función escalón definida para los valores $1, -1$.

$$f = \begin{cases} 1 & si\ w^T x > 0 \\ -1 & si\ w^T x < 0 \end{cases}$$

donde $x_0 = 1$.

Este modelo permite la implementación de un clasificador binario. La principal aportación del perceptrón es que define el primer método de aprendizaje supervisado. Los adaptación de los pesos se realiza en función del error entre la salida de la red y la salida que debería obtenerse.

Perceptrón monocapa: La regla del perceptrón

- Los pesos sinápticos y umbral se obtienen mediante un proceso adaptativo que consiste en comenzar con unos valores iniciales aleatorios e ir modificándolos iterativamente cuando la salida de la unidad no coincide con la salida deseada.
- Para modificar los pesos se sigue **la regla del perceptrón** que viene dada por la siguiente expresión:

$$w_i(k+1) = w_i(k) + \Delta w_i(k) \; para \; k = 1, 2, 3...$$

donde k es la iteración, W_i es el peso sináptico i y $\Delta w_i(k)$ se define como:

$$\Delta w_i(k) = \eta(k)[t_i - y_i(k)]x_i(k)$$

- Esta regla indica que la variación del peso w_i es proporcional al producto del error $t_i - y_i(k)$ por la componente i-ésima del patrón de entrada que hemos introducido en la iteración k, es decir, x_i. Siendo $y_i(k)$ la salida predicha de la instancia i en la iteración k y t_i la salida deseada para la entrada de la instancia i.
- La constante de proporcionalidad $\eta(k)$ es la tasa de aprendizaje que cuanto mayor es más se modifica el peso sináptico y viceversa. Este parámetro controla el proceso de aprendizaje.

Perceptrón monocapa: Algoritmo de aprendizaje

Algoritmo de aprendizaje. Entrada: Instancias X.

0. Inicializar los pesos sinápticos con números aleatorios del intervalo $[-1, 1]$.
1. Desde k=1 hasta máximo iteraciones o condición de parada. Calcular:

$$y(k) = \sum_{i=1}^{D} w_i x_i + w_0$$

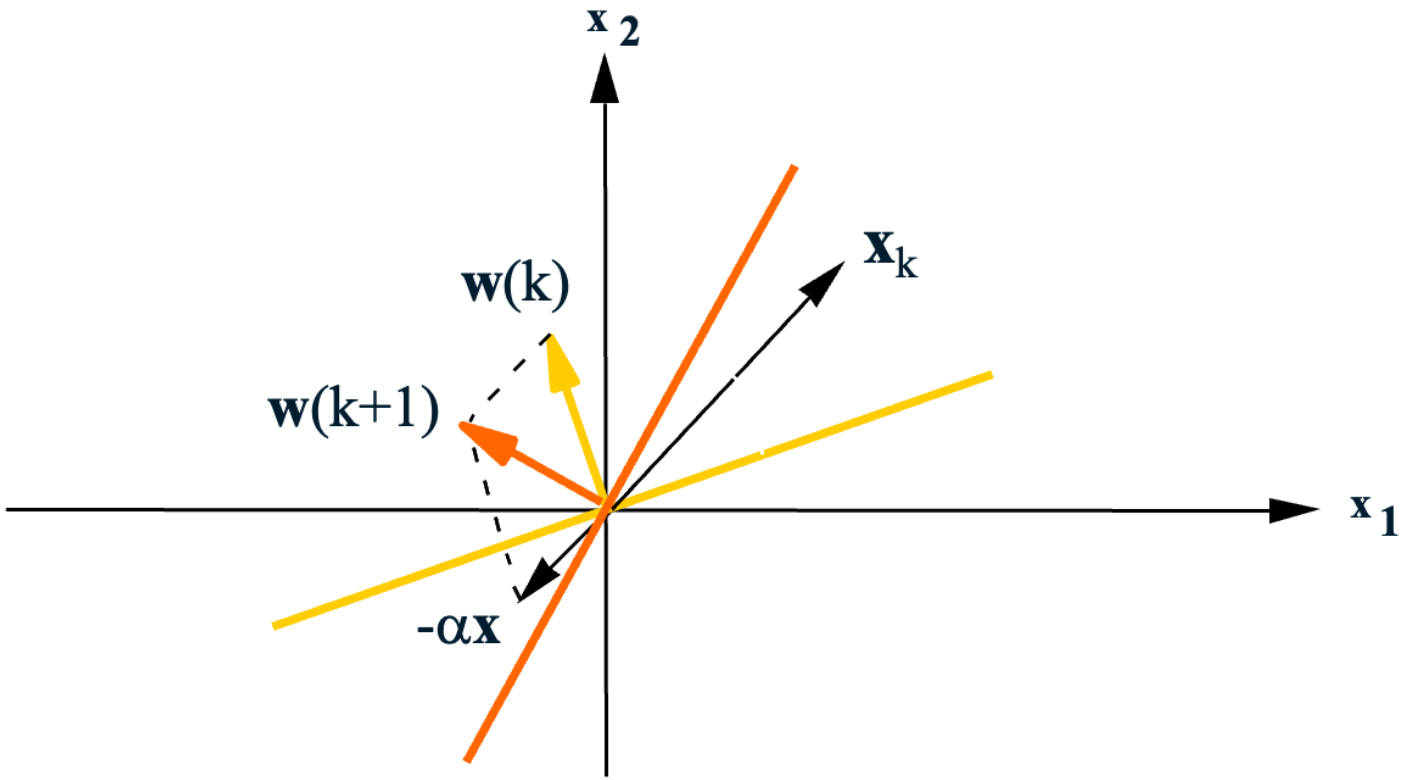
2. Corregir los pesos sinápticos. Si $y(k) \neq t$ entonces calcular: $w_i(k+1)$
3. Si no se han modificado los pesos en últimas p iteraciones, entonces condición de parada. Si no entonces volver al paso 1

¿El algortimo del perceptron puede entonces clasificar cualquier problema?

Perceptrón monocapa: Limitaciones

Solo converge si hay separabilidad lineal

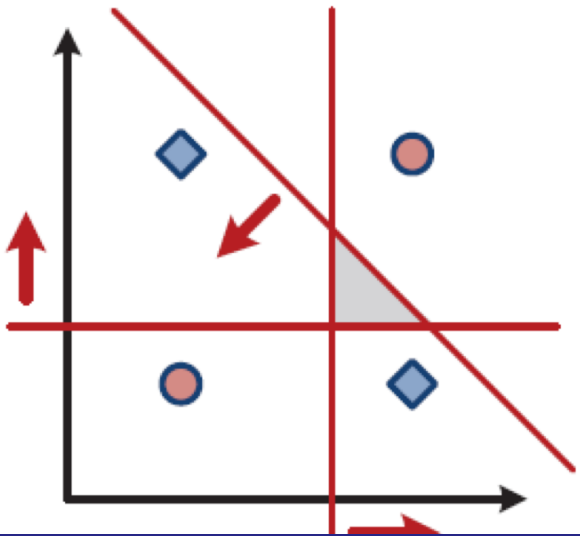
Ejemplo gráfico: $D = 2$, $w_0 = b = 0$ y una muestra/instancia $\{\mathbf{x}_k, d_k = -1\}$ y $\alpha = \eta$



Perceptrón monocapa: Limitaciones

Minsky y Papert definieron las limitaciones del perceptrón.

- Los valores de salida solo pueden ser binarios. Sólo sirve para problemas de clasificación binaria.
- Sólo puede clasificar vectores linealmente separables. Las clases están separadas por un hiperplano.
- El teorema de convergencia garantiza que, para cualquier conjunto de datos linealmente separable, la regla de aprendizaje finaliza en un número finito
  de pasos



Perceptrón monocapa: Regla de entrenamiento Delta

El problema de convergencia se puede evitar utilizando otra regla de entrenamiento. Widrow y Hopf proponen minimizar el coste cuadrático Least Mean Square (LMS) utilizando el gradiente descendiente. Se trata de determinar los pesos sinápticos de manera que se minimice la función de error cuadrático siguiente:

$$E = \frac{1}{2}\sum_{d=1}^{X}(t_d - y_d)^2$$

siendo X el conjunto de instancias de entrenamiento y d la instancia y j el atributo j de la instancia d.
El vector gradiente es:

$$\Delta E(\overrightarrow{w}) = [\frac{\partial E}{\partial w_0}, \frac{\partial E}{\partial w_1}, .., \frac{\partial E}{\partial w_n}]$$

Perceptrón monocapa: Regla de entrenamiento Delta

La regla gradiente es:

$$\overrightarrow{w} \leftarrow +\Delta\overrightarrow{w}, \quad \Delta w_i = -\eta \Delta E,$$

$$\Delta E = \frac{\partial E}{\partial w_i} = \sum_{d=0}^{X} (t_d - y_d)(-x_{dj})$$

$$\Delta w_i = \eta \sum_{d=0}^{X} (t_d - y_d) x_{dj}$$

siendo x_{dj} el valor del atributo j de la instancia d.

Así la actualización de pesos queda como:

$$w_i(k+1) = w_i + \Delta w_i$$

Perceptrón monocapa: Algoritmo del gradiente descendiente

Algoritmo del gradiente descendiente

1. Inicializar cada w_i a un valor aleatorio.
2. Desde $k = 1$ hasta máximo iteraciones o condición de parada.
   - $\Delta w_i = 0$
   - Para cada instancia x de X
     - Realizar una pasada de $\overrightarrow{x}$ y calcular y_x
     - Para cada w_i

$$\Delta w_i \leftarrow \Delta w_i + \eta(t_x - y_x)x_i$$

   - Para cada w_i

$$w_i \leftarrow w_i + \Delta w_i$$

siendo t_x la salida deseada de la instancia x y y_x la salida predicha.

Perceptrón monocapa: Regla de entrenamiento Delta

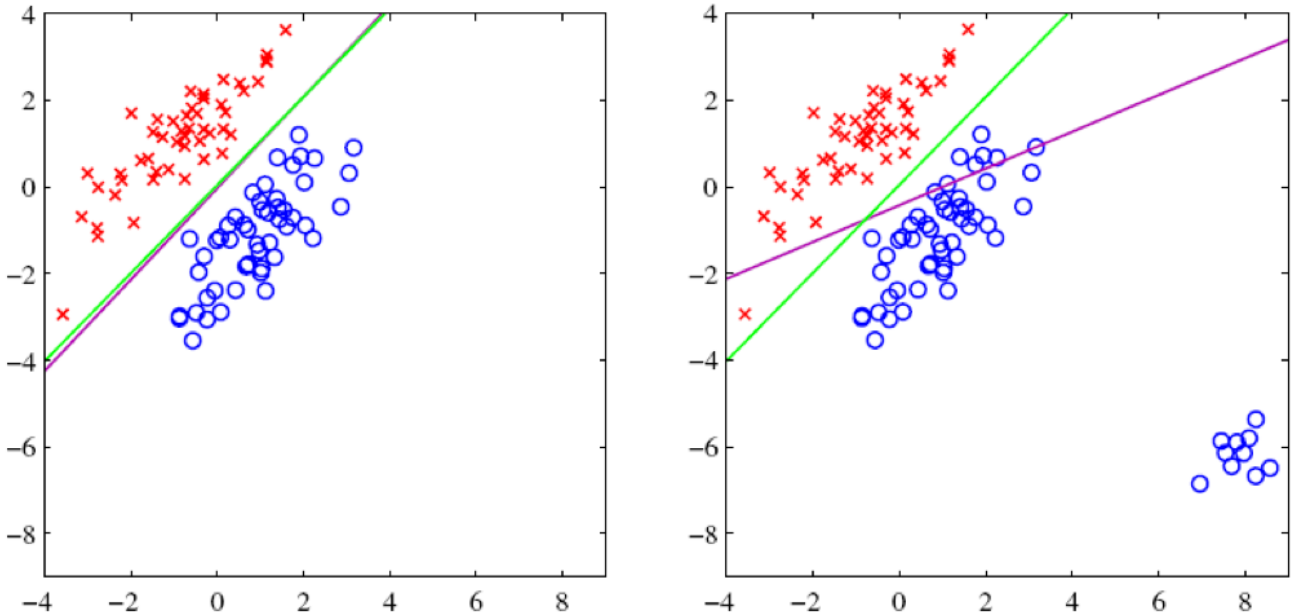
Regla de entrenamiento Delta:

- Se basa en el LMS que es muy robusto: converge en muchos casos
- Basada en gradiente descendente
- Se ajusta asintóticamente a la representación deseada
- Aprendizaje:
  - **supervisado**
  - **lineal:** se emplea la salida ($\mathbf{z}$) en el error
  - Por prestaciones: según coste cuadrático
- Produce resultados razonables (los mejores mediante una frontera lineal) aunque el problema no sea separable linealmente

Perceptrón monocapa: LMS

Problema con los "OUTLIERS"

Error cuadrático para clasificación



La figura muestra el resultado obtenido por el LMS (en morado) para dos situaciones distintas.

Perceptrón monocapa: LMS

Ahora con LMS, podemos cambiar la función de activación escalón (activación duro) por otra función de activación (activación blanda) $f'(z_k)$. Por ejemplo por una función logística, tangente o gaussiana.

$$\mathbf{w}(k+1) = \mathbf{w}(k) + \Delta w f'(z_k)\mathbf{x}_j,$$

Con esta activación podremos conseguir resultados diferentes y obtener un mejor porcentaje de clasificación. Hay que tener en cuenta que el perceptrón sólo es "válido" para clasificar cuando las clases son linealmente separables.

Perceptrón Multicapa: Introducción

- Las limitaciones de las redes de una sola capa llevó al planteamiento de más arquitecturas de Redes Neuronales Artificiales (RNA) complejas con más de una capa intermedia entre la entrada y la salida.
- La función de dichas capas intermedias es la de realizar una transformación de los datos de entrada de forma que sean linealmente separables.
- De esta forma podemos implementar cualquier función con el grado de precisión deseado.
- Así surge el Perceptrón Multicapa (MLP): una RNA que incluye una o varias capas intermedias (capas ocultas).

Perceptrón Multicapa: Introducción

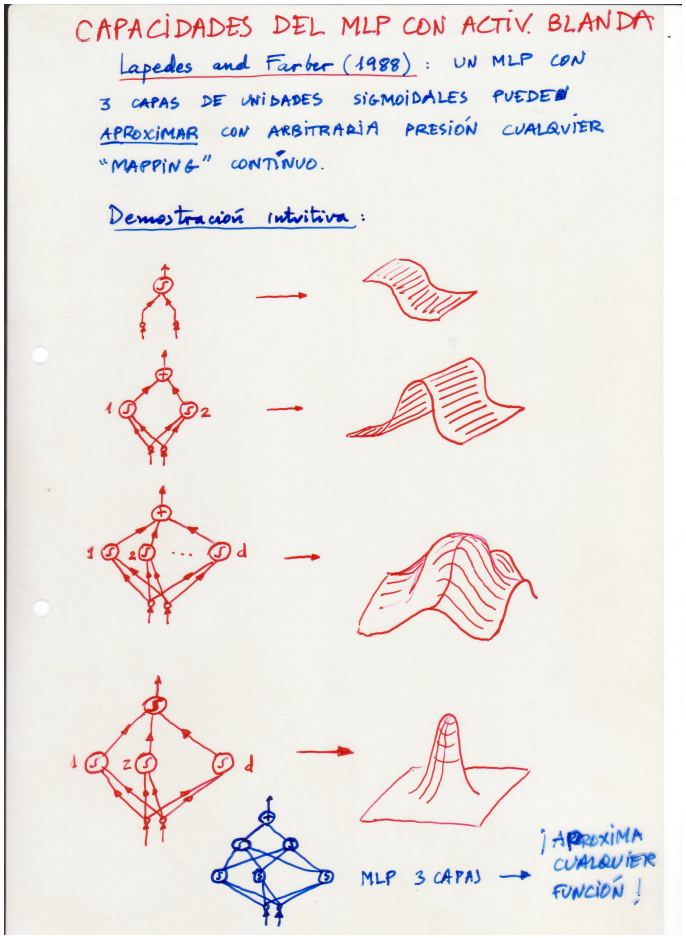
Las propiedades de las MLP son:

- potentes
- versátiles
- distribuidas: robustas
- paralelas: rápidas (entrenadas)

pero:

- de entrenamiento difícil y lento
- de difícil análisis

Perceptrón Multicapa: Capacidades



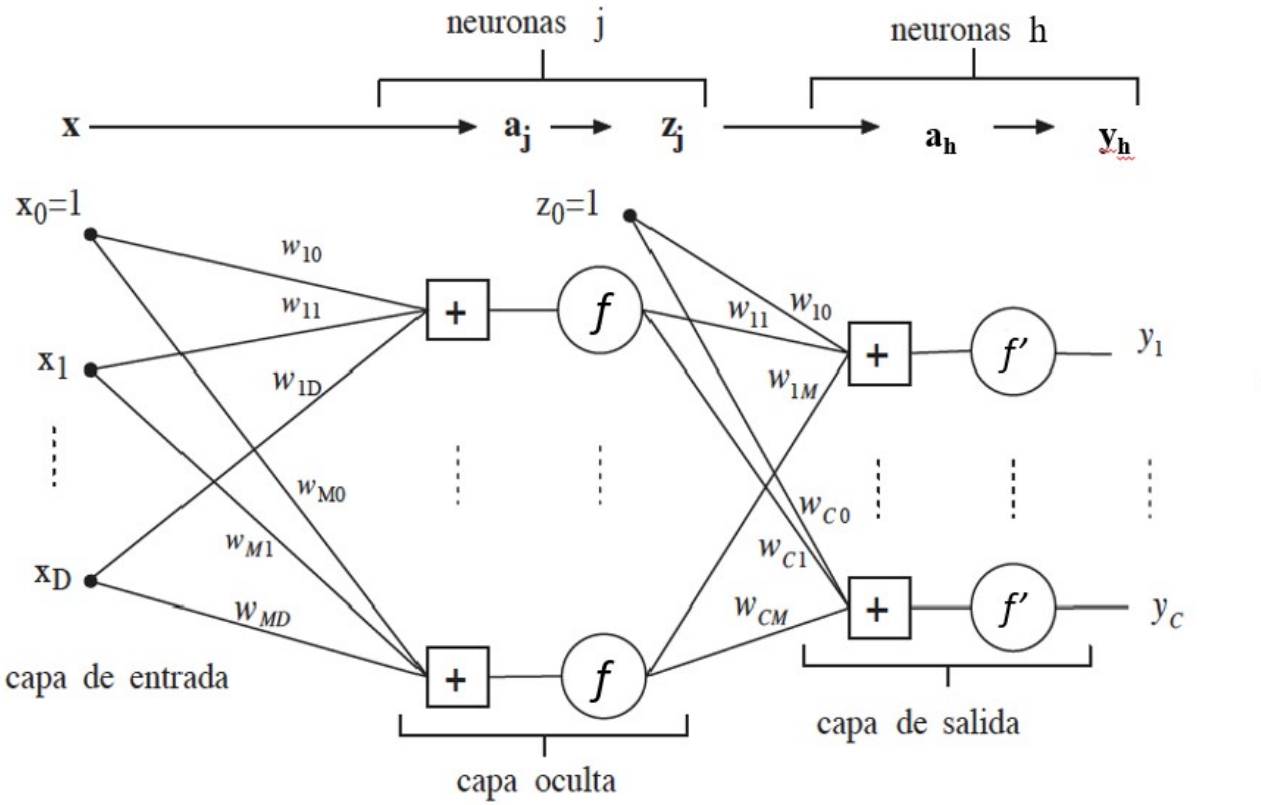
¡Son aproximadores universales!

Estructura	Regiones de Decisión	Problema de la XOR	Clases con Regiones Mezcladas	Formas de Regiones más Generales
1 Capa	Medio Plano Limitado por un Hiperplano			
2 Capas	Regiones Cerradas o Convexas			
3 Capas	Complejidad Arbitraria Limitada por el Número de Neuronas			

Perceptrón Multicapa: Arquitectura

MLP

La arquitectura consta de una seria de D neuronas de entrada, una capa oculta o más capas ocultas y una capa de salida con C neuronas de salida.



Perceptrón Multicapa: Arquitectura

Se han probado los siguientes teoremas:

Cybenko: basta con una capa oculta de unidades sigmoidales (en número indefinido) para

$$R^D \rightarrow (1,-1)^C \quad \text{(Clasificación)}$$

donde D y C son la dimensión de la entrada y la salida, respectivamente.

Kolmogorov (adaptado por Hetch-Nielsen): basta con una capa oculta de $2D+1$ unidades de activaciones adecuadas para

$$(1,-1)^D \rightarrow R^C \quad \text{(Regresión)}$$

Perceptrón Multicapa: Algoritmo de retropropagación (BP)

El MLP basa el aprendizaje de los pesos en una regla de ajuste del error.

- Algoritmo de retropropagación (backpropagation)(BP): Primero se ajustan los pesos de la capa de salida para después propagar el error a las capas anteriores hasta llegar a la capa de entrada.
- Básicamente consiste en dos fases:
  - Primero se aplica un patrón a las entradas de la red y su efecto se propaga a través de la misma, capa a capa, hasta la salida.
  - Después, se recalculan los pesos de acuerdo con la regla de ajuste del error (bakpropagation phase). Es decir, se calcula el valor de la función de error que compara la respuesta actual de la red y la respuesta deseada y este error se propaga hacia atrás. El error se calcula como:

$$E(\mathbf{w}) = \frac{1}{2} \sum_{x \in X} \sum_{o \in Y} (t_{ox} - y_{ox})^2$$

donde t_{ox} es la salida esperada para la instancia x en el nodo o, y_{ox} es la salida obtenida en el nodo o para la instancia x.

Perceptrón Multicapa: Algoritmo de retropropagación (BP)

Algoritmo de backpropagation. Entrada: Instancias X.

1. Inicializar los pesos sinápticos con números aleatorios pequeños.
2. Escoger un patrón de entrada X_i e introducirlo a la capa de entrada
3. Propagar la entrada por la red hacia adelante inicializando las neuronas de entrada con X_i y calcular los y empezando por las neuronas de la capa oculta, y después de la capa de salida y calcular el error a retropropagar.
4. Calcular los valores δ para las capas de salida como:

$$\delta_h = (T_h - Y_h)\dot{f}'(Act_h)$$

5. Calcular los valores δ para las capas oculta como:

$$\delta_j = (T_j - Y_j)\dot{f}'(Act_j)$$

6. Actualizar los pesos de acuerdo con:

$$W = W + \Delta W \qquad \Delta W = \eta \delta_i x_{ij}$$

7. Repetir del 2 al 6 para todos los patrones de entrada.

$f'(Act_h)$ es la derivada de la función de activación

Perceptrón Multicapa: Algoritmo de retropropagación (BP)

Calculo de los pesos de la siguiente iteración empleando gradiente descendiente (Werbos, Parker y Rumelhart)

$$w_{ji}^{(l)}(k+1) = w_{ji}^{(l)}(k) - \eta \frac{\partial E}{\partial w_{ji}^{(l)}}(k)$$

siendo l la capa donde se actualiza el peso.

Similitud con regla delta:

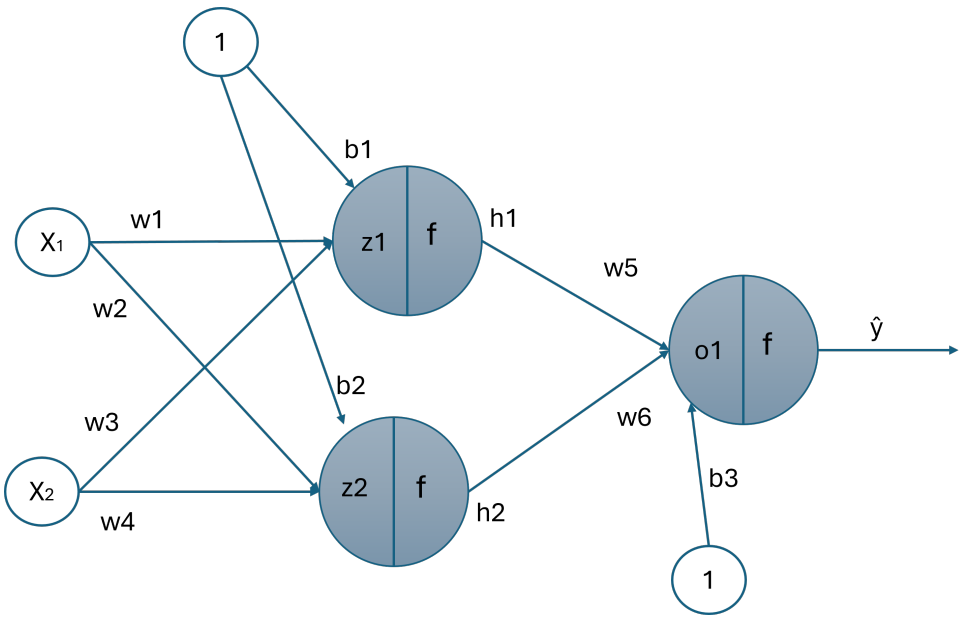
- En regla delta, w_i se actualizan con $\mathbf{w}(k+1) = \mathbf{w}(k) + \eta \sum_{d=1}^{X} (t_d - y_d)\mathbf{x}_i$.
- En backpropagation se suma a los w_{ji} el producto $\eta x_{ij} \delta_i$.

Perceptrón Multicapa: Algoritmo de retropropagación (BP)

Ejemplo: Supongamos que tenemos la siguiente configuración y considerando los siguientes valores de entrada e inicialización:

$$\eta = 0.1, X = \{0.2, 0.5\}, W = \{0.5, 0.1, 0.15, 0.4, 0.6, 0.25\}, B = \{0.1, 0.15, 0.2\},$$

$$f = RELU(x) = max(0, x), f'(x) = \begin{cases} 0 & cuando \ \ x \leq 0 \\ 1 & cuando \ \ x > 0 \end{cases}$$



Expresiones para Forward:

$$z_1 = w_1 x_1 + w_3 x_2 + b_1$$

$$z_2 = w_2 x_1 + w_4 x_2 + b_2$$

$$h_1 = f(z_1) = RELU(z_1)$$

$$h_2 = f(z_2) = RELU(z_2)$$

$$o_1 = w_5 h_1 + w_6 h_2 + b_3$$

$$\hat{y} = f(o_1) = RELU(o_1)$$

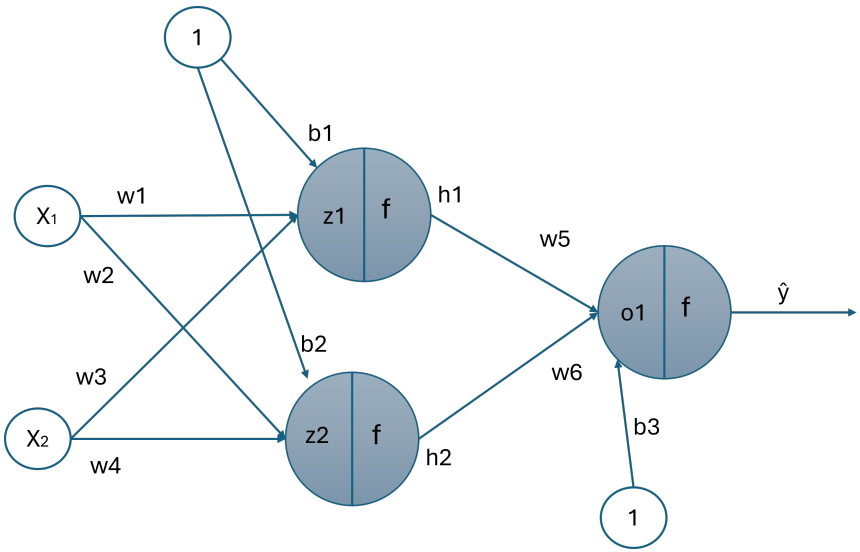
$$E = \frac{1}{2}(\hat{y} - y)^2$$

Perceptrón Multicapa: Algoritmo de retropropagación (BP)

Ejemplo: Supongamos que tenemos la siguiente configuración y considerando los siguientes valores de entrada e inicialización:

$$\eta = 0.1, X = \{0.2, 0.5\}, W = \{0.5, 0.1, 0.15, 0.4, 0.6, 0.25\}, B = \{0.1, 0.15, 0.2\},$$

$$f = RELU(x) = max(0, x), f'(x) = \begin{cases} 0 & cuando \ \ x \leq 0 \\ 1 & cuando \ \ x > 0 \end{cases}$$



Cálculo de Forward:

$$z_1 = 0.5 \cdot 0.2 + 0.15 \cdot 0.5 + 0.1 = 0.85$$

$$z_2 = 0.1 \cdot 0.2 + 0.4 \cdot 0.5 + 0.15 = 0.37$$

$$h_1 = f(z_1) = 0.85$$

$$h_2 = f(z_2) = 0.37$$

$$o_1 = 0.6 \cdot 0.85 + 0.25 \cdot 0.37 + 0.2 = 0.803$$

$$\hat{y} = f(o1) = 0.803$$

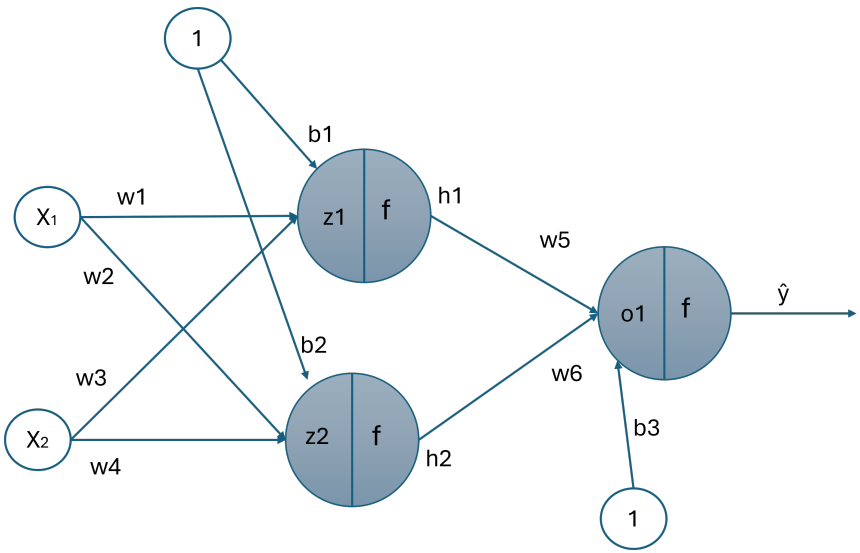
$$E = \frac{1}{2}(0.803 - 0)^2 = 0.322$$

Perceptrón Multicapa: Algoritmo de retropropagación (BP)

Ejemplo: Supongamos que tenemos la siguiente configuración y considerando los siguientes valores de entrada e inicialización:

$$\eta = 0.1, X = \{0.2, 0.5\}, W = \{0.5, 0.1, 0.15, 0.4, 0.6, 0.25\}, B = \{0.1, 0.15, 0.2\},$$

$$f = RELU(x) = max(0, x), f'(x) = \begin{cases} 0 & cuando \ \ x \leq 0 \\ 1 & cuando \ \ x > 0 \end{cases}$$



Cálculo Backward: Aplicamos la regla de la cadena, primero capa de salida:

$$w_5^{t+1} = w_5^t - \eta \frac{\partial E}{\partial w_5}$$

$$\frac{\partial E}{\partial w_5} = \frac{\partial E}{\partial \hat{y}} \frac{\partial \hat{y}}{\partial o_1} \frac{\partial o_1}{\partial w_5}$$

$$\frac{\partial E}{\partial \hat{y}} = \frac{\partial \frac{1}{2}(y - \hat{y})^2}{\partial \hat{y}} = -(y - \hat{y}) = \hat{y} - y$$

$$\frac{\partial \hat{y}}{\partial o_1} = \frac{\partial RELU(o_1)}{\partial o_1}$$

$$\frac{\partial o_1}{\partial w_5} = h_1$$

$$w_5^{t+1} = w_5^t - \eta(\hat{y} - y) \frac{\partial RELU(o_1)}{\partial o_1} h_1$$

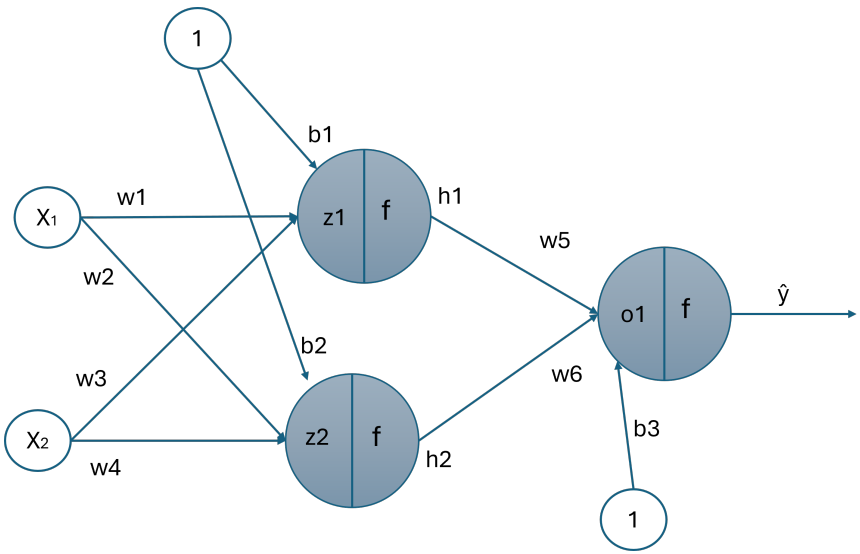
$$w_5^{t+1} = 0.6 - 0.1 \cdot (0.0 - 0) \cdot 1 \cdot 0.85 = 0.6 - 0.068 = 0.531$$

Perceptrón Multicapa: Algoritmo de retropropagación (BP)

Ejemplo: Supongamos que tenemos la siguiente configuración y considerando los siguientes valores de entrada e inicialización:

$$\eta = 0.1, X = \{0.2, 0.5\}, W = \{0.5, 0.1, 0.15, 0.4, 0.6, 0.25\}, B = \{0.1, 0.15, 0.2\},$$

$$f = RELU(x) = max(0, x), f'(x) = \begin{cases} 0 & cuando \ \ x \leq 0 \\ 1 & cuando \ \ x > 0 \end{cases}$$



Cálculo Backward: Aplicamos la regla de la cadena, primero pesos de la capa de salida:

$$w_6^{t+1} = w_6^t - \eta(\hat{y} - y)\frac{\partial RELU(o_1)}{\partial o_1} h_2$$

$$w_6^{t+1} = 0.25 - 0.1 \cdot (0.8 - 0) \cdot 1 \cdot 0.37 = 0.2204$$

Actualización del sesgo para la capa de salida:

$$b_3^{t+1} = b_3^t - \eta \frac{\partial E}{\partial b_3}$$

$$\frac{\partial E}{\partial b_3} = \frac{\partial E}{\partial \hat{y}} \frac{\partial \hat{y}}{\partial o_1} \frac{\partial o_1}{\partial b_3} = \frac{\partial E}{\partial \hat{y}} \frac{\partial \hat{y}}{\partial o_1}$$

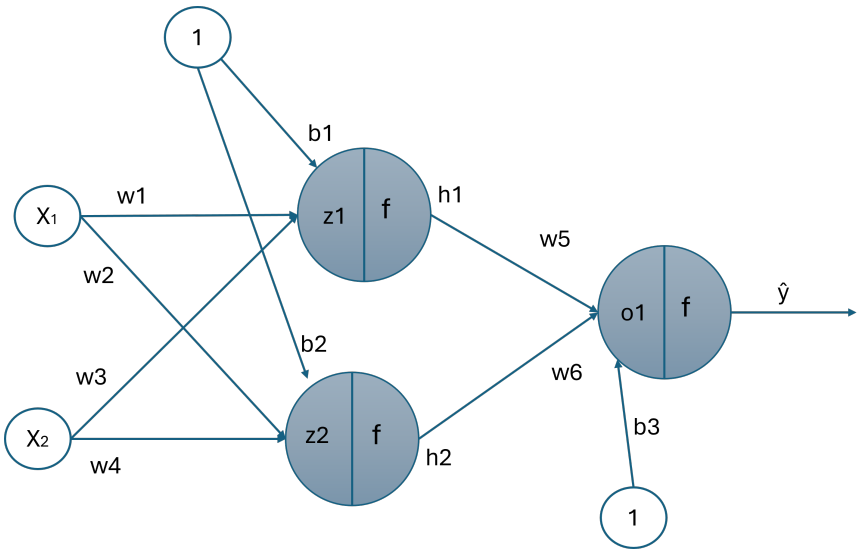
$$b_3 = 0.2 - 0.1 \cdot (0.8 - 0) = 0.12$$

Perceptrón Multicapa: Algoritmo de retropropagación (BP)

Ejemplo: Supongamos que tenemos la siguiente configuración y considerando los siguientes valores de entrada e inicialización:

$$\eta = 0.1, X = \{0.2, 0.5\}, W = \{0.5, 0.1, 0.15, 0.4, 0.6, 0.25\}, B = \{0.1, 0.15, 0.2\},$$

$$f = RELU(x) = max(0, x), f'(x) = \begin{cases} 0 & cuando \ \ x \leq 0 \\ 1 & cuando \ \ x > 0 \end{cases}$$



Cálculo Backward: Aplicamos la regla de la cadena, ahora pesos de la capa de oculta:

$$w_1^{t+1} = w_1^t - \eta \frac{\partial E}{\partial w_1} \qquad w_3^{t+1} = w_3^t - \eta \frac{\partial E}{\partial w_3}$$

$$\frac{\partial E}{\partial w_1} = \frac{\partial E}{\partial \hat{y}} \frac{\partial \hat{y}}{\partial o_1} \frac{\partial o_1}{\partial h_1} \frac{\partial h_1}{\partial z_1} \frac{\partial z_1}{\partial w_1}$$

$$\frac{\partial E}{\partial w_1} = (\hat{y} - y) \frac{\partial RELU(o_1)}{\partial o_1} w_5 \frac{\partial RELU(z_1)}{\partial z_1} x_1$$

$$w_1^{t+1} = 0.15 - 0.1 \cdot (0.8 - 0) \cdot 1 \cdot 0.6 \cdot 1 \cdot 0.2 = 0.1404$$

$$\frac{\partial E}{\partial w_3} = \frac{\partial E}{\partial \hat{y}} \frac{\partial \hat{y}}{\partial o_1} \frac{\partial o_1}{\partial h_1} \frac{\partial h_1}{\partial z_1} \frac{\partial z_1}{\partial w_3}$$

$$\frac{\partial E}{\partial w_3} = (\hat{y} - y) \frac{\partial RELU(o_1)}{\partial o_1} w_5 \frac{\partial RELU(z_1)}{\partial z_1} x_2$$

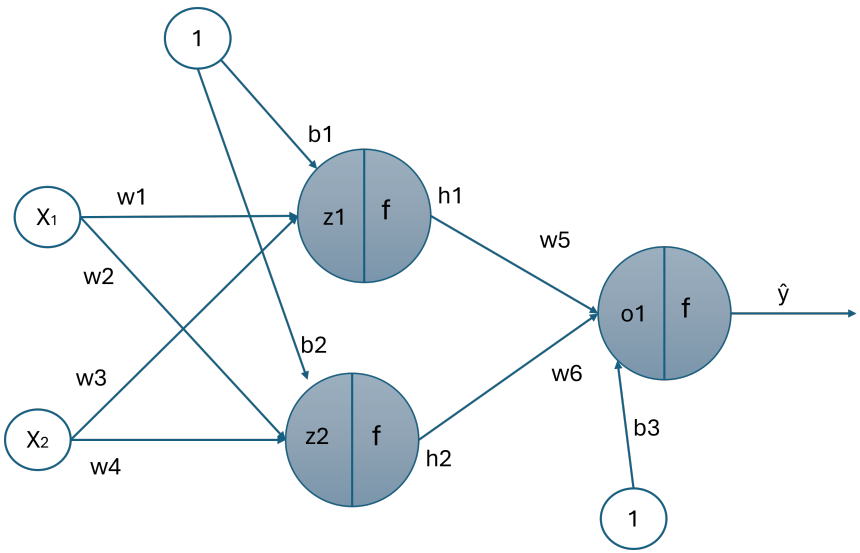
$$w_3^{t+1} = 0.15 - 0.1 \cdot (0.8 - 0) \cdot 1 \cdot 0.6 \cdot 1 \cdot 0.5 = 0.126$$

Perceptrón Multicapa: Algoritmo de retropropagación (BP)

Ejemplo: Supongamos que tenemos la siguiente configuración y considerando los siguientes valores de entrada e inicialización:

$$\eta = 0.1, X = \{0.2, 0.5\}, W = \{0.5, 0.1, 0.15, 0.4, 0.6, 0.25\}, B = \{0.1, 0.15, 0.2\},$$

$$f = RELU(x) = max(0, x), f'(x) = \begin{cases} 0 & cuando \ \ x \leq 0 \\ 1 & cuando \ \ x > 0 \end{cases}$$



Cálculo Backward: Aplicamos la regla de la cadena, ahora pesos de la capa de oculta:

$$w_2^{t+1} = w_2^t - \eta \frac{\partial E}{\partial w_2} \qquad w_4^{t+1} = w_4^t - \eta \frac{\partial E}{\partial w_4}$$

$$\frac{\partial E}{\partial w_2} = \frac{\partial E}{\partial \hat{y}} \frac{\partial \hat{y}}{\partial o_1} \frac{\partial o_1}{\partial h_2} \frac{\partial h_2}{\partial z_2} \frac{\partial z_2}{\partial w_2}$$

$$\frac{\partial E}{\partial w_2} = (\hat{y} - y) \frac{\partial RELU(o_1)}{\partial o_1} w_6 \frac{\partial RELU(z_2)}{\partial z_2} x_1$$

$$w_2^{t+1} = 0.1 - 0.1 \cdot (0.8 - 0) \cdot 1 \cdot 0.25 \cdot 1 \cdot 0.2 = 0.096$$

$$\frac{\partial E}{\partial w_4} = \frac{\partial E}{\partial \hat{y}} \frac{\partial \hat{y}}{\partial o_1} \frac{\partial o_1}{\partial h_1} \frac{\partial h_2}{\partial z_2} \frac{\partial z_2}{\partial w_4}$$

$$\frac{\partial E}{\partial w_4} = (\hat{y} - y) \frac{\partial RELU(o_1)}{\partial o_1} w_6 \frac{\partial RELU(z_2)}{\partial z_2} x_2$$

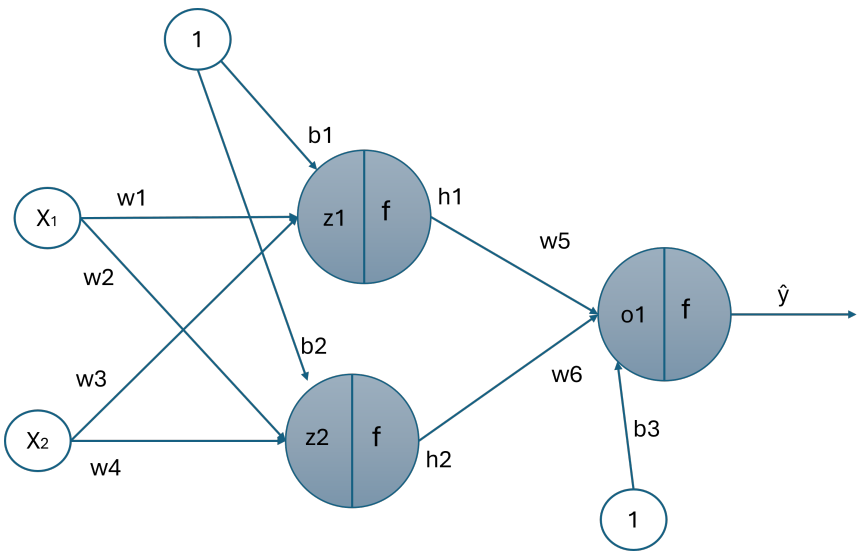
$$w_2^{t+1} = 0.4 - 0.1 \cdot (0.8 - 0) \cdot 1 \cdot 0.25 \cdot 1 \cdot 0.5 = 0.39$$

Perceptrón Multicapa: Algoritmo de retropropagación (BP)

Ejemplo: Supongamos que tenemos la siguiente configuración y considerando los siguientes valores de entrada e inicialización:

$$\eta = 0.1, X = \{0.2, 0.5\}, W = \{0.5, 0.1, 0.15, 0.4, 0.6, 0.25\}, B = \{0.1, 0.15, 0.2\},$$

$$f = RELU(x) = max(0, x), f'(x) = \begin{cases} 0 & cuando \ \ x \leq 0 \\ 1 & cuando \ \ x > 0 \end{cases}$$



Actualización del sesgo para la capa oculta:

$$b_1^{t+1} = b_1^t - \eta \frac{\partial E}{\partial b_1} \qquad b_2^{t+1} = b_2^t - \eta \frac{\partial E}{\partial b_2}$$

$$\frac{\partial E}{\partial b_1} = \frac{\partial E}{\partial \hat{y}} \frac{\partial \hat{y}}{\partial o_1} \frac{\partial o_1}{\partial h_1} \frac{\partial h_1}{\partial z_1} \frac{\partial z_1}{\partial b_1}$$

$$\frac{\partial E}{\partial b_1} = (\hat{y} - y) \frac{\partial RELU(o_1)}{\partial o_1} w_5 \frac{\partial RELU(z_1)}{\partial z_1}$$

$$b_1^{t+1} = 0.1 - 0.1 \cdot (0.8 - 0) \cdot 1 \cdot 0.6 \cdot 1 = 0.052$$

$$\frac{\partial E}{\partial b_2} = \frac{\partial E}{\partial \hat{y}} \frac{\partial \hat{y}}{\partial o_1} \frac{\partial o_1}{\partial h_2} \frac{\partial h_2}{\partial z_2} \frac{\partial z_2}{\partial b_2}$$

$$\frac{\partial E}{\partial b_2} = (\hat{y} - y) \frac{\partial RELU(o_1)}{\partial o_1} w_5 \frac{\partial RELU(z_2)}{\partial z_2}$$

$$b_2^{t+1} = 0.15 - 0.1 \cdot (0.8 - 0) \cdot 1 \cdot 0.25 \cdot 1 = 0.13$$

Perceptrón Multicapa: Algoritmo de retropropagación (BP)

Condiciones de parada

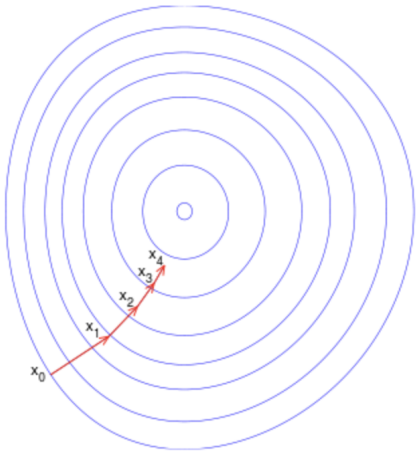
- En un problema "real" se realizan usualmente varios miles de pasadas (i.e. epochs) por todo el conjunto de entrenamiento.
- A veces la condición de parada del algoritmo es simplemente llegar a un número límite de esas pasadas
- Otras es no alcanzar una cantidad mínima de error por pasada.
- Se ha de evitar el sobreaprendizaje.

Perceptrón Multicapa: BP - Modos de entrenamiento

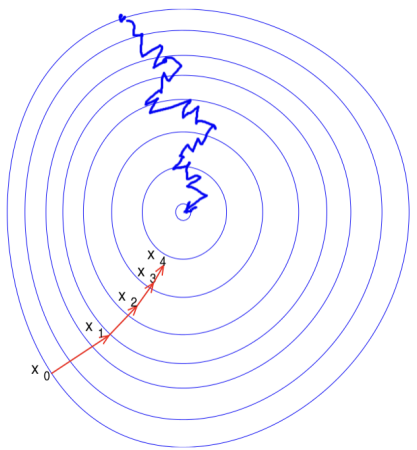
¿Cada cuánto se actualizan los pesos?

- Batch: después de cada época del conjunto de entrenamiento
- Mini-Batch: después de una instancia (conjunto de patrones) del conjunto de entrenamiento
- Online: después de cada ejemplo

Modo **batch**



Modo **estocástico** (mini-batch y online)

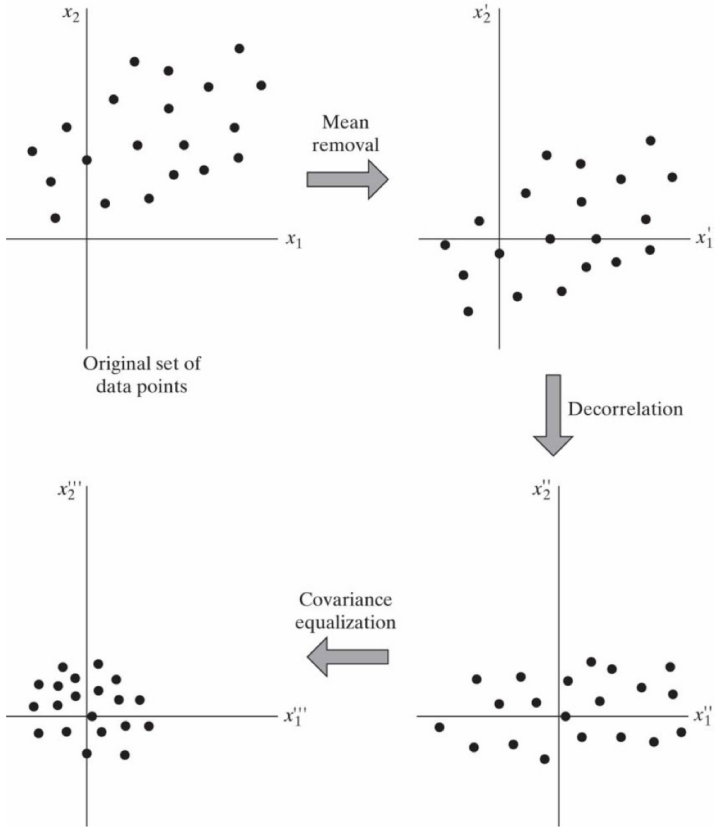


Perceptrón Multicapa: BP - Instancias de entrada

- Conjunto de muestras/instancias/ejemplos de entrenamiento representativo
- Conviene preprocesar las muestras para eliminar información que de seguro se sabe irrelevante
- Conviene normalizar (características con media nula y varianza unidad)
- Pueden ser útiles códigos sencillos (en general las redes neuronales muestran sensibilidad al formato de presentación). Ejemplos: 1:N, termómetro, etc.
- Aleatorizar el orden de presentación de las muestras y ciclar las series de entrenamiento. Se evita la inestabilidad temporal

Perceptrón Multicapa: BP - Instancias de entrada

- Decorrelar las entradas. Un método para decorrelar las variables de entrada: **Análisis de Componentes Principales (PCA)**
  - Permite reducir la dimensionalidad de los datos (eliminando componentes con menores autovalores)
  - Permite convertir una superficie de error elíptica en una circular, en la que el gradiente apunta directamente al mínimo (dividiendo los componentes principales por las raíces cuadradas de sus respectivos autovalores)



Si no se aplican bien estos principios pueden aparecer problemas de convergencia.

Perceptrón Multicapa: BP - Configuración de parámetros

No hay reglas fijas, sino empíricas

En un principio, un número aconsejable de capas ocultas es:

- 1 ó 2 para clasificación
- 2 para regresión

(No contradice los teoremas de Cybenko y Kolmogorov: 2 pueden dar lugar a una arquitectura más eficiente)
Hoy día, el **Deep Learning** permite el empleo de muchas más capas ocultas.

Inicialización de parámetros y variables

- Para entradas normalizadas:
  - η entre 0.001 y 1
  - Pesos iniciales aleatorios con distribución $U[-0.5, 0.5]$

Perceptrón Multicapa: BP - Configuración de parámetros

Mejora de la **convergencia**

- **Modificación de la tasa de aprendizaje**:
  - **Descenso de la tasa de aprendizaje** (Learning Rate Decay): Comienza con una tasa de aprendizaje (parámetro de aprendizaje o **escalón**) alta y luego disminúyela gradualmente durante el entrenamiento. Esto permite un rápido avance al principio y una convergencia más suave hacia el mínimo global a medida que avanzas.
  - **Tasa de aprendizaje adaptativa**: Utiliza algoritmos como Adam, RMSprop o Adagrad que ajustan automáticamente la tasa de aprendizaje en función del historial de gradientes. Estos algoritmos pueden converger más rápido en muchos casos. Idea básica:
    - Si la convergencia al mínimo va bien (disminuye el error), se aumenta el escalón
    - Si la convergencia al mínimo va mal (aumenta el error), se vuelve atrás y se disminuye el escalón

Perceptrón Multicapa: BP - Configuración de parámetros

Soluciones generales ante la LENTITUD

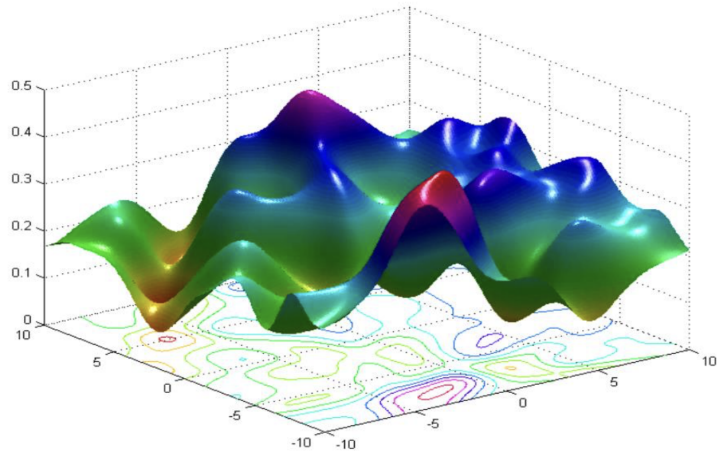
- Modificación o cambio de la **función de coste**
- Gestión del **parámetro de aprendizaje**
- Técnicas de **regularización**
- Utilización de otros **algoritmos de búsqueda**
- Métodos de **selección de muestras/instancias/ejemplos**

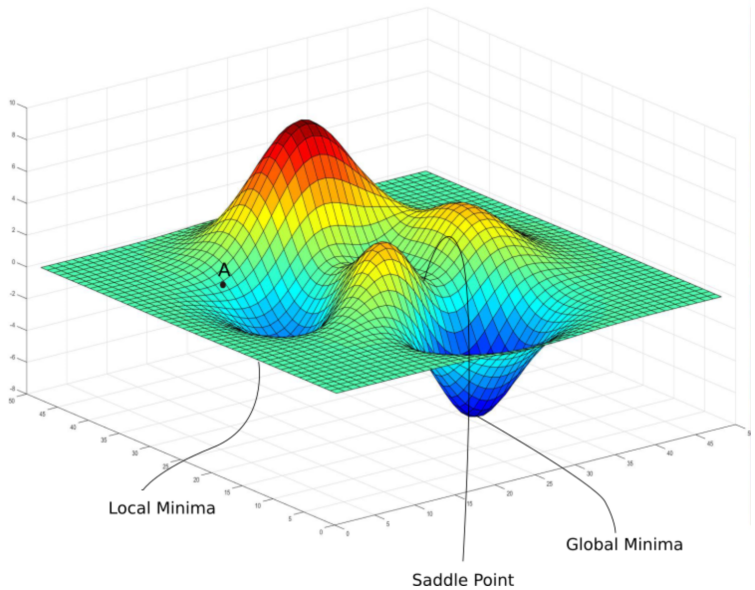
Perceptrón Multicapa: BP - Convergencia

Problemas de convergencia: **Mínimos locales**

Para evitar ser atrapados por ellos:

- Método sencillo: **varias inicializaciones** de los pesos, quedándonos con el mejor resultado
- **Métodos naturales**:
  - **Neo-Darwinianos**: Genéticos, Evolutivos, etc.
  - **Temple simulado**
  - Perforación de **túneles**

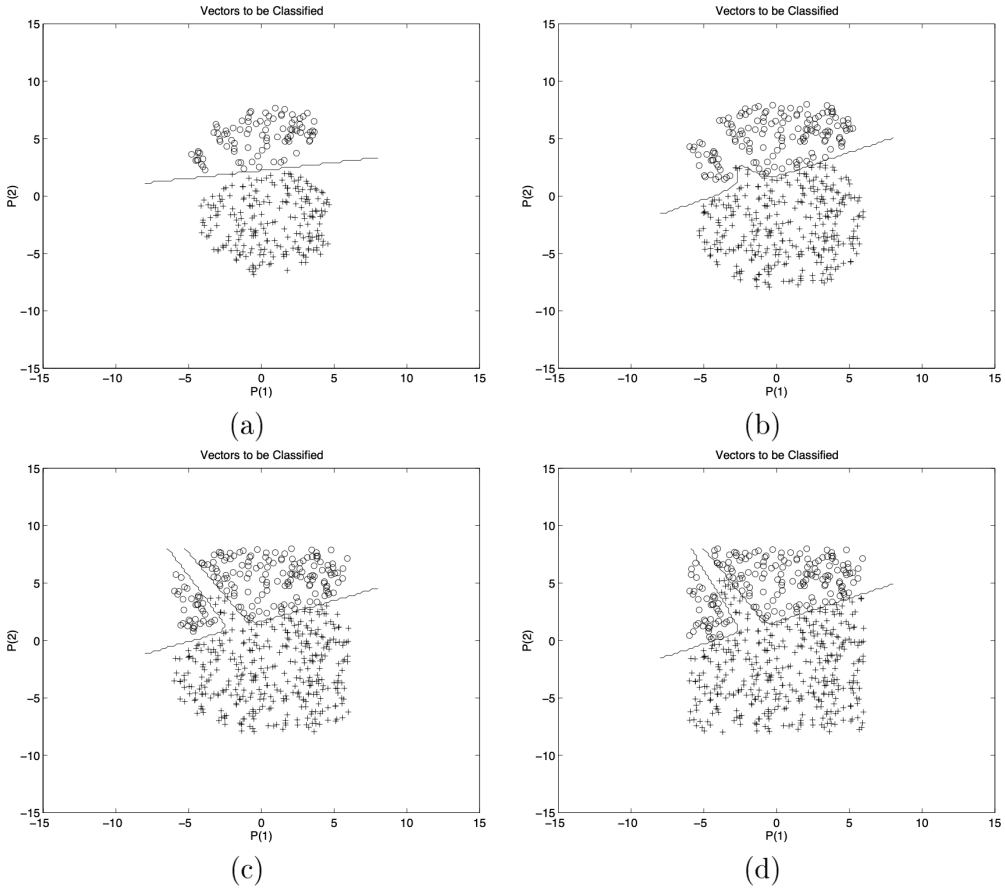




Perceptrón Multicapa: BP - Convergencia

Problemas de convergencia: **Mínimos locales**

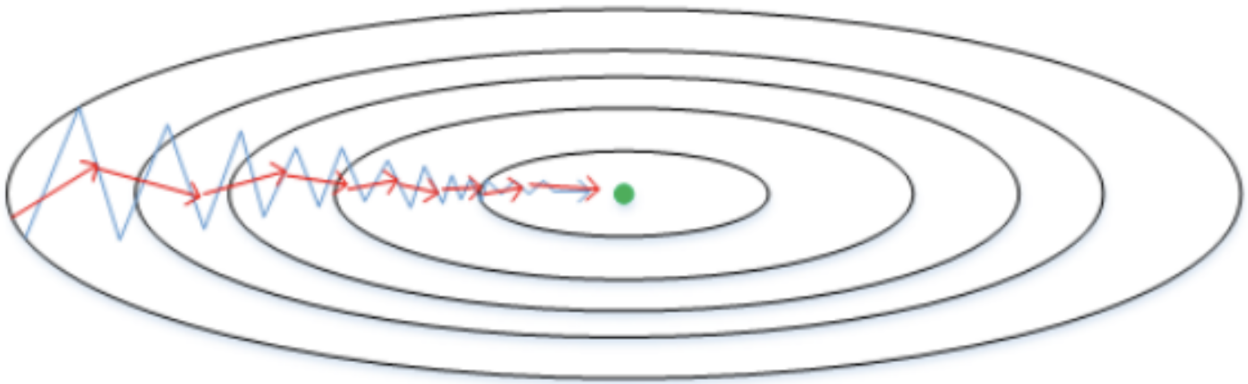
- **Aprendizaje incremental** mediante selección de muestras. Garantizan o facilitan llegar al mínimo local (con más carga computacional)



Perceptrón Multicapa: BP - Convergencia

En la convergencia al mínimo, lo que se busca es:

- Movernos más rápido en direcciones con gradientes pequeños pero consistentes (que apuntan al mínimo)
- Movernos más despacio en direcciones con gradientes grandes pero inconsistentes



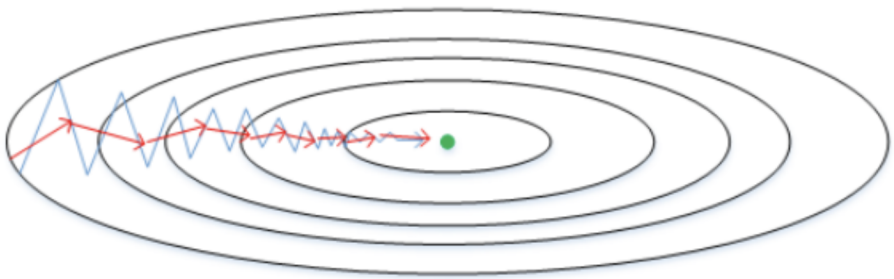
Perceptrón Multicapa: BP - Momento

Mejora de la **convergencia**

- **Método del Momento (Momentum** (modificación del gradiente):

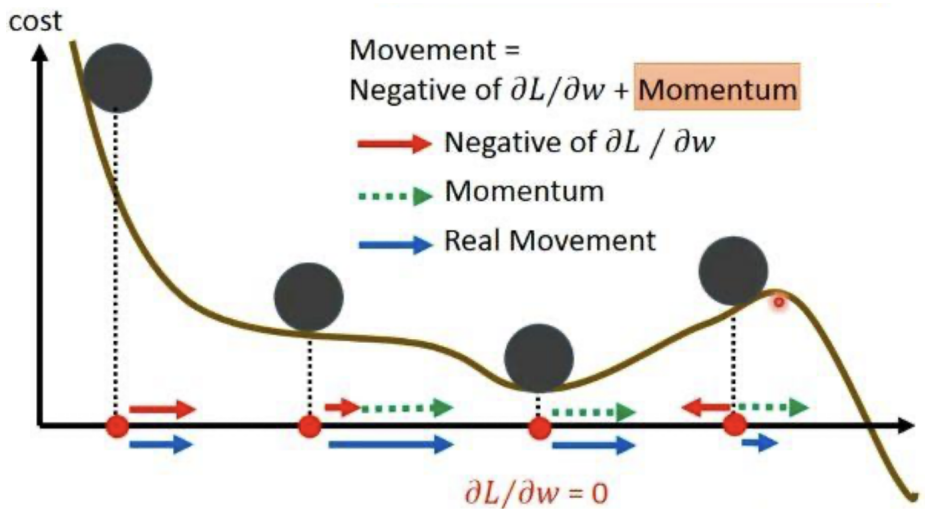
$$w_{ji}^{(l)}(k+1) = w_{ji}^{(l)}(k) - \eta \frac{\partial E}{\partial w_{ji}^{(l)}}(k) + \mu \left(w_{ji}^{(l)}(k) - w_{ji}^{(l)}(k-1) \right)$$

  - Se amortiguan oscilaciones en direcciones de alta curvatura combinando gradientes de signo contrario.
  - Se aumenta la velocidad en direcciones con un gradiente pequeño pero consistente.



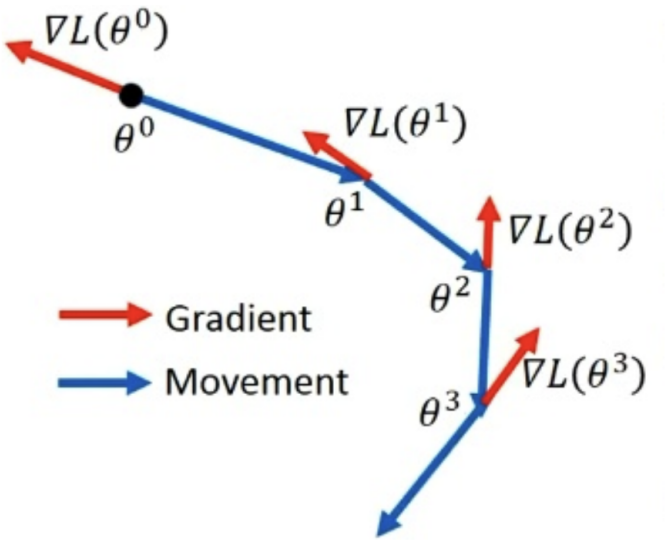
Perceptrón Multicapa: BP - Momento

El momento añade inercia al movimiento; útil en llanuras.

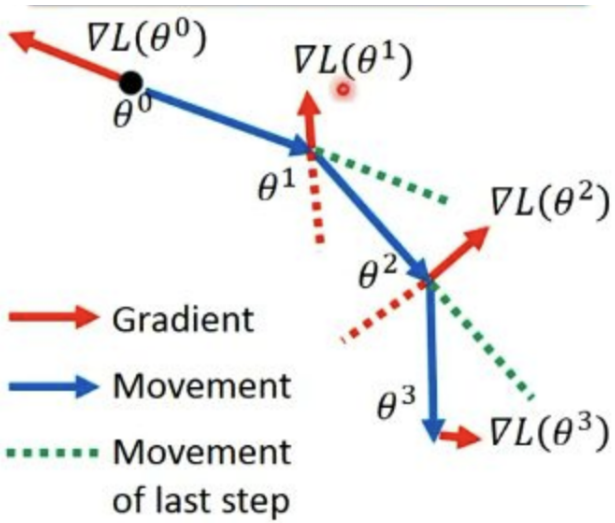


Perceptrón Multicapa: BP - Momento

Movimiento según **gradiente**



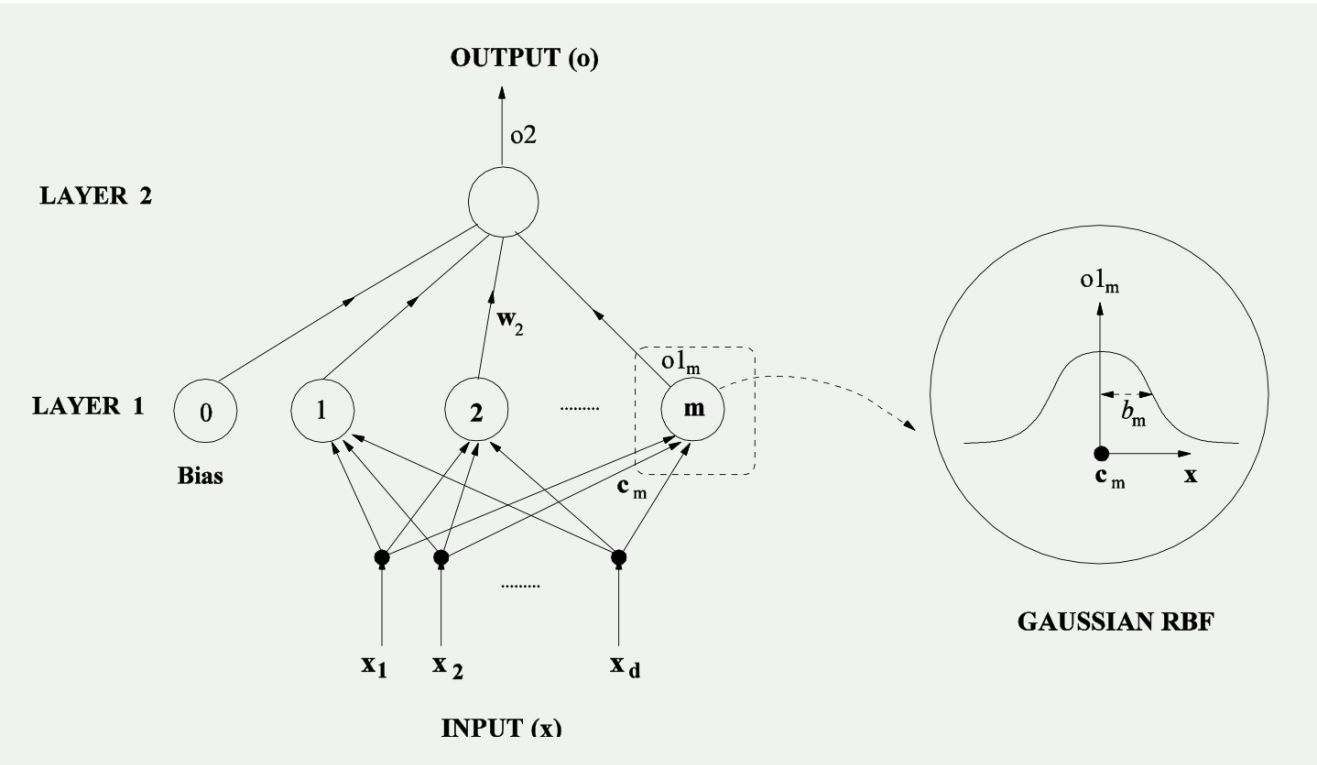
Movimiento según **gradiente con momento**



Redes de funciones de base radial

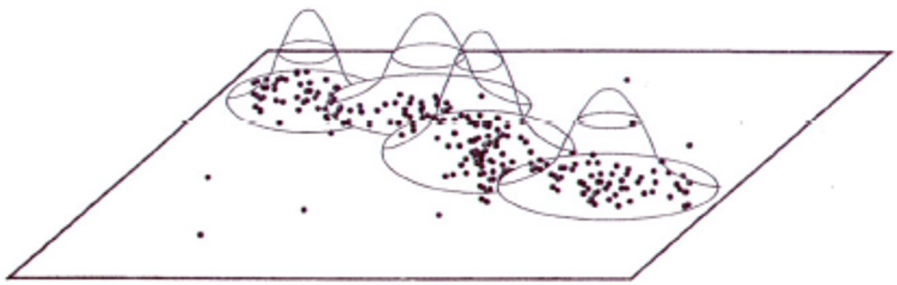
Son redes multicapa y aproximadores universales

Realizan un mapeado no lineal en su primera capa. Se caracteriza por tener un entrenamiento híbrido. La arquitectura de estas redes se compone de tres capas: una de entrada, una única capa oculta y una capa de salida.



Redes de base radial

$$\begin{pmatrix} d_1 \\ d_2 \\ \vdots \\ d_N \end{pmatrix} = \begin{pmatrix} \phi(\mathbf{x}_1, \mathbf{c}_1) & \phi(\mathbf{x}_1, \mathbf{c}_2) & \dots & \phi(\mathbf{x}_1, \mathbf{c}_m) \\ \phi(\mathbf{x}_2, \mathbf{c}_1) & \phi(\mathbf{x}_2, \mathbf{c}_2) & \dots & \phi(\mathbf{x}_2 \mathbf{c}_m) \\ \vdots & \vdots & \ddots & \vdots \\ \phi(\mathbf{x}_N, \mathbf{c}_1) & \phi(\mathbf{x}_N, \mathbf{c}_2) & \dots & \phi(\mathbf{x}_N, \mathbf{c}_m) \end{pmatrix} \begin{pmatrix} w_1 \\ w_2 \\ \vdots \\ w_m \end{pmatrix} \quad (1)$$



- Park, J. and Sandberg, I. W. (1991). "Universal Approximation using Radial-Basis-Function Networks", Neural Computation, 3(2), 246-257.
- Park, J. and Sandberg, I. W. (1993). .Approximation and Radial-Basis-Function Networks," Neural Computation, 5(2), 305-316.

Redes de base radial: Propuestas de RBF

Primera propuesta: **interpolación exacta**

(Boomhead and Lowe, 1988)

$$F(\mathbf{x}) = \sum_{i=1}^{N} w_i \varphi \|\mathbf{x} - \mathbf{c}_i\|$$

Con **tantos centroides como muestras/instancias**, es decir

$$\mathbf{c}_i = \mathbf{x}_i \qquad \rightarrow \qquad F(\mathbf{x}) = \sum_{i=1}^{N} w_i \varphi \|\mathbf{x} - \mathbf{x}_i\|$$

Entrenamiento para conseguir: $F(\mathbf{x}) = t_i$, siendo t_i la salida deseada

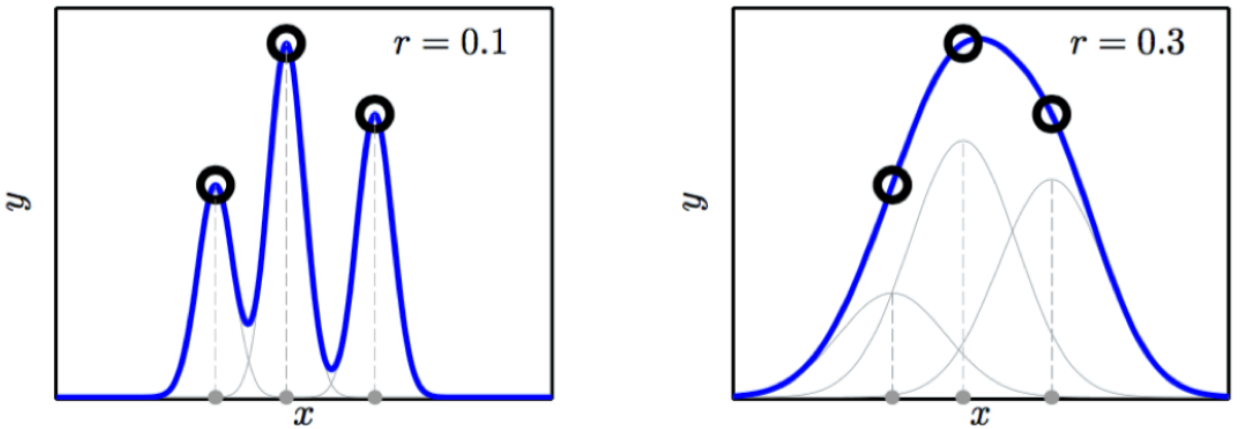
$$\mathbf{t} = \mathbf{\Phi}\mathbf{w} \qquad \rightarrow \qquad \mathbf{w} = \mathbf{\Phi}^{-1}\mathbf{t}$$

Normalmente, la interpolación perfecta es mala solución

Redes de base radial: Propuestas de RBF

Ejemplo de interpolación exacta

Problema de regresión con núcleos gaussianos de dos anchuras diferentes.



(c) Parametric RBF, small r (b) Parametric RBF, large r

Para un problema de clasificación binaria, basta multiplicar las gausianas por la salida deseada (+1, -1) de la instancia correspondiente.

Redes de base radial: Propuestas de RBF

Nueva propuesta

(Poggio y Girosi, 1990)

$$F(\mathbf{x}) = \sum_{i=1}^{m} w_i \varphi \|\mathbf{x} - \mathbf{c}_i\|$$

donde $m < N$ siendo N número total de instancias.

Redes de base radial: Funciones radiales

- Gausiana regular (la más usada):

$$\varphi_i(\mathbf{x}) = \exp\left(\frac{1}{2\sigma_i^2}\|\mathbf{x} - \mathbf{c}_i\|\right)$$

- Gausiana:

$$\varphi_i(\mathbf{x}) = \exp\left(-\frac{1}{2}(\mathbf{x} - \mathbf{c}_i)^T \Sigma_i^{-1}(\mathbf{x} - \mathbf{c}_i)\right)$$

  No muy usados por compensar el aumento de prestaciones con el computacional.

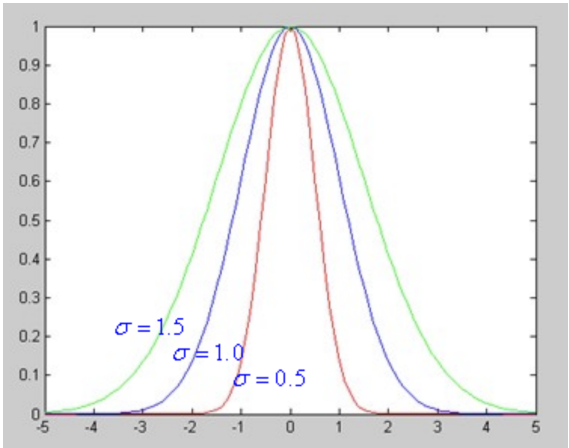
- Multicuadrática inversa

$$\varphi_i(\mathbf{x}) = \frac{c}{\sqrt{\|\mathbf{x} - \mathbf{c}_i\|^2 + c^2}}, \qquad c > 0$$

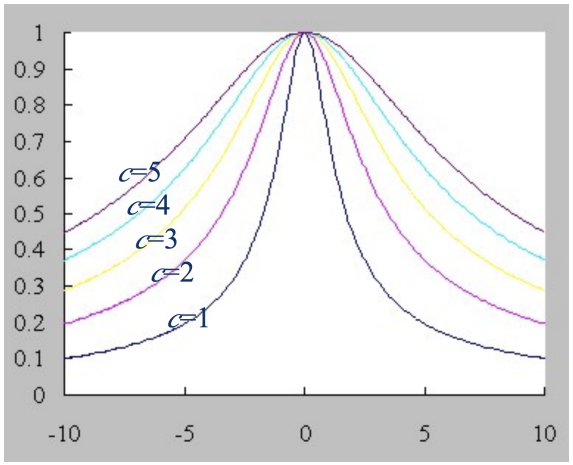
  donde c es una constante (distinta que los centroides $\mathbf{c}_i$).

Redes de base radial: Funciones radiales

Guasiana



Multicuadrática inversa



Redes de base radial: Entrenamiento

Método 1. Centroides fijados aleatoriamente

- En el entrenamiento de una RBF con centroides aleatorios, se inicializan los centroides de las funciones de base radial de manera aleatoria.
- Luego, se ajustan los parámetros de las funciones de base radial (como la varianza) y los pesos de la capa de salida utilizando algún algoritmo de entrenamiento, como el método de la pseudoinversa o el de mínimos cuadrados por gradiente.

Redes de base radial: Entrenamiento

Método 1. Centroides fijados aleatoriamente

1) Se fija la varianza:

$$\boxed{\sigma = \frac{y_{max}^2}{2m}}, \qquad \text{donde} \quad y_{max} = \max_{j,i} \|\mathbf{x}_j - \mathbf{c}_i\|, \quad j = 1, \ldots, N \quad i = 1, \ldots, m$$

2) Se calculan los pesos para que se cumpla: $\mathbf{y} = \mathbf{\Phi w}$

$$\begin{pmatrix} y_1 \\ y_2 \\ \vdots \\ y_N \end{pmatrix} = \begin{pmatrix} \varphi(\mathbf{x}_1, \mathbf{c}_1) & \varphi(\mathbf{x}_1, \mathbf{c}_2) & \ldots & \varphi(\mathbf{x}_1, \mathbf{c}_m) \\ \varphi(\mathbf{x}_2, \mathbf{c}_1) & \varphi(\mathbf{x}_2, \mathbf{c}_2) & \ldots & \varphi(\mathbf{x}_2 \mathbf{c}_m) \\ \vdots & \vdots & \ddots & \vdots \\ \varphi(\mathbf{x}_N, \mathbf{c}_1) & \varphi(\mathbf{x}_N, \mathbf{c}_2) & \ldots & \varphi(\mathbf{x}_N, \mathbf{c}_m) \end{pmatrix} \begin{pmatrix} w_1 \\ w_2 \\ \vdots \\ w_m \end{pmatrix}$$

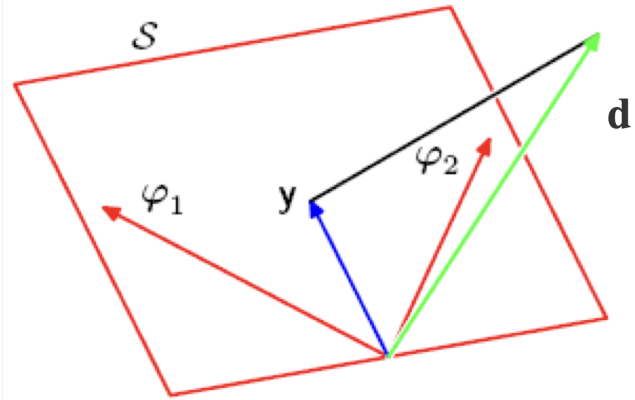
Solución: Normalmente, la solución exacta no existe (sistema sobredeterminado). Sí existe solución óptima en términos de error cuadrático (SSE).

Redes de base radial: Entrenamiento

Pseudoinversa de Moore-Penrose

La solución óptima analítica SSE viene dada por el vector $\mathbf{w}$ que minimiza el error cuadrático entre la salida ($\mathbf{y}$) y los targets ($\mathbf{t}$), es decir

$$E(\mathbf{w}) = \frac{1}{2}\sum_{i=1}^{N}(t_n - y_n)^2 = \frac{1}{2}\|\mathbf{t} - \mathbf{y}\|^2,$$



Su expresión es $\boxed{\mathbf{w}^* = \mathbf{\Phi}^+\mathbf{y}}$ donde Φ^+ es la **pseudoinversa** de Moore-Penrose dada por $\Phi^+ = (\Phi^T\Phi)^{-1}\Phi^T$.

La salida producida por la solución óptima, $\mathbf{y} = \mathbf{\Phi}\mathbf{w}^*$, es la proyección de $\mathbf{d}$ sobre el subespacio S generado por las columnas de Φ, minimizándose la distancia euclidea entre $\mathbf{t}$ e $\mathbf{y}$.

Mínimos cuadrados por gradiente

Por ejemplo, mediante el algoritmo LMS (ya visto).

Redes de base radial: Entrenamiento

Método 2. Centroides fijados según cuantificación vectorial

- En este entrenamiento, se inicializan los centroides de las funciones de base radial mediante el empleo de algún método de cuantificación vectorial (VQ)
- Luego, se ajustan los parámetros de las funciones de base radial (como la varianza) y los pesos de la capa de salida utilizando algún algoritmo de entrenamiento, como el método de la pseudoinversa o el de mínimos cuadrados.

Algunos ejemplos de método VQ son:

- Algoritmo FSCL ("Frequency Sensitive Competitive Learning")
- Mapas autoorganizados
- Clustering. Ej.: Algoritmo de K-medias.

Redes de base radial: Entrenamiento

Método 3. Entrenamiento completo por gradiente supervisado

Todas los pesos se entrenan por gradiente para minimizar la función de error.

$$E = \frac{1}{2}\sum_{j=1}^{N} e_j^2 \quad \text{donde} \quad e_j = d_j - \sum_{i=1}^{m} \omega_i \varphi(\|\mathbf{x}_j - \mathbf{c}_i\|)$$

$$\mathbf{c}_k(m+1) = \mathbf{c}_k(m) - \eta_1 \frac{\partial E(m)}{\partial c_k(m)} \qquad \frac{\partial E(m)}{\partial c_k(m)} = 2\omega_k(m) \sum_{j=1}^{N} e_j(m)\varphi'(\|\mathbf{x}_j - \mathbf{c}_k(m)\|)\Sigma_k^{-1}[\mathbf{x}_j - \mathbf{c}_k(m)]$$

$$\omega_k(m+1) = \omega_k(m) - \eta_2 \frac{\partial E(m)}{\partial \omega_k(m)} \qquad \frac{\partial E(m)}{\partial \omega_k(m)} = \sum_{j=1}^{N} e_j(m)\varphi(\|\mathbf{x}_j - \mathbf{c}_k(m)\|)$$

$$\Sigma_k^{-1}(m+1) = \Sigma_k^{-1}(m) - \eta_3 \frac{\partial E(m)}{\partial \Sigma_k^{-1}(m)} \qquad \frac{\partial E(m)}{\partial \Sigma_k^{-1}(m)} = -\omega_k(m) \sum_{j=1}^{N} e_j(m)\varphi'(\|\mathbf{x}_j - \mathbf{c}_k(m)\|)\mathbf{Q}_{jk}(m)$$

$$\mathbf{Q}_{jk}(m) = [\mathbf{x}_j - \mathbf{c}_k(m)][\mathbf{x}_j - \mathbf{c}_k(m)]^T$$

Bibliografía relacionada I

- José Hernández Orallo, M.José Ramírez Quintana, Cèsar Ferri Ramírez, 2004. Introducción a la Minería de Datos. Pearson education. Editorial Pearson, 2004. ISBN: 84 205 4091 9
- Jiawei Han, Micheline Kamber. *Data Mining: Concepts and Techniques*. Morgan Kaufman Publishers, 2003.
- Christopher Bishop. *Pattern Recognition and Machine Learning*. Springer-Verlag, 2006
- L. A. Breslow and D. W. Aha. *Simplifying decision trees: a survey*. Knowledge Engineering Review, 12(1):1–40, 1997.
- Sheerama K. Murthy. *Automatic construction of decision trees from data: A multi-disciplinary survey*. Data Mining and Knowledge Discovery, 1997.
- Ethem Alpaydin. *Introduction to Machine Learning*. MIT Press 2004.
- José T. Palma y Roque Marín (eds.). *Inteligencia Artificial: Métodos, Técnicas y Aplicaciones*. McGraw-Hill, 2008